



RAFFLES INSTITUTION

2026 YEAR 6 PRELIMINARY EXAM

CANDIDATE
NAME

CLASS

26

COMPUTING

9569/02

Paper 2 (Lab based)

August 2026

3 hours

Additional materials:

- Electronic version of WAREHOUSE1.txt data file
- Electronic version of WAREHOUSE2.txt data file
- Electronic version of REGISTRATIONS.txt data file
- Electronic version of WORKSHOPS.db file
- Electronic version of REGISTER.html file
- Electronic version of REPORT.html file
- Insert Quick Reference Guide

READ THESE INSTRUCTIONS FIRST

Answer **all** questions.

All tasks must be done in the computer laboratory. You are not allowed to bring in or take out any pieces of work or materials on paper or electronic media or in any other form.

Approved calculators are allowed.

Save each task as it is completed.

The use of built-in functions, where appropriate, is allowed for this paper unless stated otherwise.

Note that up to **6** marks will be awarded for the use of common coding standards for programming style.

The number of marks is given in brackets [] at the end of each task.

The total number of marks for this paper is 100.

FOR EXAMINER'S USE				
Task 1	Task 2	Task 3	Task 4	
				TOTAL
				100

This document consists of **12** printed pages.

RAFFLES INSTITUTION
Mathematics Department

Instructions to candidates:

Your program code and output for each of Task 1, 2 and 4 should be saved in a single `.ipynb` file using Jupyter Notebook. For example, your program code and output for Task 1 should be saved as:

`TASK1_<your name>_<centre number>_<index number>.ipynb`

1 Name your Jupyter Notebook as:

`TASK1_<your name>_<centre number>_<index number>.ipynb`

A warehouse uses an autonomous cleaning robot to clean the floor after working hours. The warehouse floor is represented by a two-dimensional array named `Warehouse`. Each element of the array contains one of the following characters.

Character	Meaning
.	Dirty floor
C	Clean floor
#	Shelf (obstacle)
R	Robot

Initially there is exactly one robot in the warehouse.
An example warehouse is shown below.

```

. . . . . . . . .
. # . . . . . . .
. . . # . . . . .
. . . . . . # . .
. . . . R . . . .
. . . . . . . . .
. # . . . . . . .
. . . . # . . . .

```

The robot may move one square at a time using these commands:

Command	Direction
U	Up
D	Down
R	Right
L	Left

The robot cannot move outside the warehouse boundaries or onto a shelf. Whenever the robot successfully moves, the square it leaves becomes clean and the robot occupies its new position.

For each of the sub-tasks, add a comment at the beginning of the code, using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

```
In
[1]:
```

```
#Task 1.1
Program code
```

Output:

Task 1.1

Write a function `LoadWarehouse(Filename)` that

- reads the warehouse map from a text file named `Filename`,
- stores the map in a two-dimensional list named `Warehouse`, with each inner list representing one row of the warehouse,
- locates the position of the robot,
- returns the `Warehouse`, the `RobotRow` and `RobotCol` in that order.

Write program code to test your function using the file `WAREHOUSE1.txt`. Display the `Warehouse` and the robot's starting position. [4]

Task 1.2

Write a function `MoveRobot(Warehouse, RobotRow, RobotCol, Direction)` where `Direction` is one of "U", "D", "R", "L" to attempt to move the robot one square in the specified direction. The robot can move onto both dirty squares and previously cleaned squares.

If the move is invalid (outside boundaries or onto a shelf),

- the function must not modify the `Warehouse`;
- it should display "Invalid move",
- and return `RobotRow` and `RobotCol`.

If the move is successful,

- the function must update the `Warehouse` to show the robot's new position,
- change the square previously occupied by the robot to 'C',
- and return the robot's new row and column.

Starting with the `Warehouse`, `RobotRow` and `RobotCol` obtained from Task 1.1, write code to test the function using the four directions in the order U, D, R and L. Update `RobotRow` and `RobotCol` after each call. Display the `Warehouse` layout after each move attempt. [6]

Task 1.3

Write a function `ScanNearby(Warehouse, RobotRow, RobotCol)` that examines only the four adjacent squares (Up, Down, Right and Left). If an adjacent position lies outside the warehouse, it should be counted as a warehouse boundary.

The function must return the following four counts in this exact order:

- Dirty floor tiles
- Clean floor tiles
- Shelves
- Warehouse boundaries

Write code to call `LoadWarehouse()` to obtain a fresh `Warehouse`, `RobotRow` and `RobotCol` for testing `ScanNearby()`, and display the four returned counts. [4]

Task 1.4

Write the necessary program code to repeatedly prompt the user to enter one of the following commands:

Command	Action
U	Move the robot up
D	Move the robot down
R	Move the robot right
L	Move the robot left
S	Scan adjacent squares
Q	Quit the simulation

Maintain two counters:

- `SuccessfulMoves`, initially 0
- `InvalidMoves`, initially 0

The program must continue until the user enters `Q`.

For each movement command:

- call `MoveRobot()` to attempt the movement,
- update the robot's current position using the values returned,
- increment `SuccessfulMoves` if the move is successful, otherwise increment `InvalidMoves`,
- display the warehouse.

If the user enters `S`, call `ScanNearby()` and display the four returned count values. Entering `S` does not affect either counter.

If an invalid command is entered, display "Invalid command" and prompt the user again.

When the simulation ends, determine the number of dirty tiles remaining from the final array state and display the summary results in this format:

```
Successful moves : xx
Invalid moves : xx
Dirty tiles remaining : xx
```

where `xx` is the appropriate value.

Call `LoadWarehouse()` using the appropriate warehouse file to obtain the `Warehouse`, `RobotRow` and `RobotCol` required to start the simulation.

Use the following command sequence to test your program with `WAREHOUSE1.txt`:

```
U U R D L L L S R U Q
```

In a new cell, call `LoadWarehouse()` using `WAREHOUSE2.txt` to initialise a new simulation. Then repeat the same command sequence. [5]

2 Name your Jupyter Notebook as:

TASK2_<your name>_<centre number>_<index number>.ipynb

A college maintains a list of students registered for its courses. The raw registration records are stored in a text file named `REGISTRATIONS.txt`.

Each student record is stored on a separate line within the text file, with the attributes separated by a comma character (,) in the following order:

Attributes	Example data
Course Code	CS101
Course Name	Introduction to Programming
Student Name	Alvin Tan
Year of Study	1

Example of a record inside `REGISTRATIONS.txt`:

`CS101,Introduction to Programming,Alvin Tan,1`

Each Course Code identifies one course. A Course Code may occur more than once because several students may be registered for the same course.

For each of the sub-tasks, add a comment at the beginning of the code, using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

```
In [1]: #Task 2.1
        Program code
```

Output:

Task 2.1

Read the data from `REGISTRATIONS.txt` into the two-dimensional list `Registrations`. Each element of `Registrations` must be a list containing one student registration.

Write a recursive function `MergeSort(Registrations, First, Last)`.

The parameters are:

- `Registrations`: the two-dimensional list of student course records.
- `First`: the integer index of the first element in the sub-list to be sorted.
- `Last`: the integer index of the last element in the sub-list to be sorted.

Sort the registrations into ascending alphabetical order by Course Code. If two or more registrations have the same Course Code, sort those registrations into ascending alphabetical order by Student Name.

Using Python's built-in sorting functions will not gain credit for this task.

Write program code to display the first ten registrations before sorting and the first ten registrations after sorting. [11]

Task 2.2

Write an iterative function `BinarySearch(Registrations, CourseCode)` that searches the sorted list for a specified Course Code.

The parameters are:

- `Registrations`: the sorted two-dimensional list of student course records.
- `CourseCode`: the Course Code to search for.

The function must

- return the index of **one** matching registration if one is found;
- return -1 if no matching registration exists.

Write program code to test your function using the Course Codes `CS101` and `CS999`. Display the index returned for each search.

If the function returns -1, display the Course Code followed by "Course not found". [4]

Task 2.3

Write a function `DisplayCourseList(Registrations, CourseCode)` to display the names of all students registered for the specified Course Code.

The parameters are:

- `Registrations`: the sorted two-dimensional list of student course records.
- `CourseCode`: the Course Code for which the student names are to be displayed.

Since the list is sorted by Course Code, all registrations for the same Course Code are stored consecutively.

Your function must:

- call `BinarySearch()` to locate a registration with the specified Course Code;
- use the index returned by `BinarySearch()` to locate and display the names of all students registered for that Course Code;
- display the Course Code followed by "Course not found" if no matching registration exists.

Do not scan the list from index 0 to the end to find the registrations for the specified Course Code. A solution that does so will not gain credit for this requirement.

Write program code to test your function using the Course Codes `CS302` and `CS999`. Display the output for each test. [5]

Task 2.4

Write a function `CourseStatistics (Registrations)` to calculate and display statistics about the course registrations.

The parameter is:

- `Registrations`: the sorted two-dimensional list of student course records.

Your function must display:

- the number of students registered for each Course Code;
- the total number of different Course Codes;
- the Course Code(s) with the largest class size.

For example,

```
CS101: 50 students
CS102: 40 students
CS201: 30 students
CS301: 20 students
```

```
Total courses: 4
```

```
Largest class:
CS101
```

If two or more Course Codes have the same largest class size, display all of them.

Write program code to test your function and display the results.

[5]

3 A college organises a number of enrichment workshops for its students.

A web application is to be created to register students for workshops and view workshop registration statistics.

The database `WORKSHOPS.db` stores the data and contains the following tables.

Student

Field	Description
StudentID	Unique student identifier
StudentName	Student name
CivicsClass	Civics class

Workshop

Field	Description
WorkshopID	Unique workshop identifier
Title	Workshop title
Category	Workshop category
MaxPlaces	Maximum number of students permitted to register

Registration

Field	Description
RegistrationID	Unique registration identifier (generated automatically)
StudentID	Student identifier
WorkshopID	Workshop identifier
RegistrationDate	Date of registration

The following files are provided:

- `WORKSHOPS.db`
- `REGISTER.html`
- `REPORT.html`

For this question, create a folder named:

`TASK3_<your name>_<centre number>_<index number>`

and save all files/subfolders created for this question in this folder.

Save your Python program as:

`TASK3_<your name>_<centre number>_<index number>.py`

For each of the sub-tasks, add a comment at the beginning of the code, using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

```
#Task 3.1
Program code
```

Task 3.1

Write a Python Flask program and create the `INDEX.html` file for the home page.

Write an SQL statement to retrieve all records from the `Workshop` table in ascending order of `Title`.

The home page must display the retrieved records. For each workshop, display `WorkshopID`, `Title`, `Category` and `MaxPlaces`.

On the home page, provide navigation links to:

- the workshop registration page
- the workshop registration report page. [4]

Run the web application.

Save the web page output as:

TASK3_1_<your name>_<centre number>_<index number>.html [1]

Task 3.2

Extend your web application from Task 3.1 to create a workshop registration page.

Use the provided file `REGISTER.html`. It contains HTML elements that display two drop-down lists.

The students list displays the `StudentID` and `StudentName` for each student. The workshops list displays the `WorkshopID` and `Title` for each workshop.

Write the necessary SQL statements to retrieve all students and workshops from the database and use the retrieved data to populate these lists.

Display the students in ascending order of `StudentID` and the workshops in ascending order of `WorkshopID`.

When the user submits the registration form, determine whether:

- the selected workshop has reached its maximum number of places; and
 - a registration already exists for the selected student and workshop.
- If either condition is true, display an appropriate message and do not insert the registration.

Otherwise, insert a new record into the `Registration` table using the current system date as the `RegistrationDate`. Display a confirmation message indicating that the registration was successful.

Save your updated Python program. [9]

Run your program and test the following cases:

- register `StudentID S011` for `WorkshopID W104` (duplicate registration);
- register `StudentID S033` for `WorkshopID W101` (workshop is full);
- register `StudentID S066` for `WorkshopID W108` (successful registration).

Save the web page output for each test case as:

```
TASK3_2_<your name>_<centre number>_<index number>_1.html
TASK3_2_<your name>_<centre number>_<index number>_2.html
TASK3_2_<your name>_<centre number>_<index number>_3.html
```

[2]

Task 3.3

Extend your web application to create a workshop registration report page.

Use the provided file `REPORT.html`. It contains the HTML elements required to display the workshop registration report.

Write an SQL statement to retrieve the `WorkshopID`, `Title`, `MaxPlaces` and number of students registered for each workshop.

Create a report showing all workshops, including workshops with no students registered.

For each workshop display:

- `WorkshopID`
- `Title`
- Maximum number of places
- Number of students registered
- Number of places remaining

Display the workshops in descending order of the number of students registered. If two or more workshops have the same number of students registered, display them in ascending order of `Title`.

At the end of the report, display the title of the workshop(s) with the highest number of students registered. If two or more workshops have the same highest number of students registered, display the title of each workshop.

Save your updated Python program. [8]

Run the web application.

Save the web page output as:

```
TASK3_3_<your name>_<centre number>_<index number>.html
```

[2]

4 Name your Jupyter Notebook as:

TASK4_<your name>_<centre number>_<index number>.ipynb

A college organises an annual Business Innovation Challenge.
Student teams present their business ideas to a panel of judges.

The class `Presentation` contains the following private attributes:

- `OrderNumber`: **Presentation order number**
- `TeamName`: **Name of the team**
- `ProjectTitle`: **Title of the business project**
- `Category`: **Competition category**
- `NextPresentation`: **Reference to the next `Presentation` object**

The class `Presentation` contains the following methods:

- `getOrderNumber()` that returns the `OrderNumber` of that object
- `getTeamName()` that returns the `TeamName` of that object
- `getProjectTitle()` that returns the `ProjectTitle` of that object
- `getCategory()` that returns the `Category` of that object
- `getNext()` that returns the `NextPresentation` of that object
- `setNext(P)` that sets `NextPresentation` to reference the `Presentation` object `P`
- `display()` that displays the `OrderNumber`, `TeamName`, `ProjectTitle` and `Category` of that object

The class `PresentationSchedule` contains one attribute:

- `FirstPresentation`: **reference to the first `Presentation` object**. Initially, `FirstPresentation = None` and the presentation schedule is empty.

The class `PresentationSchedule` contains the following methods:

- `getFirst()` that returns the first `Presentation` object
- `setFirst(P)` that sets `FirstPresentation` to reference the `Presentation` object `P`
- `AddPresentation(NewPresentation)` is a **recursive method**. It inserts `NewPresentation` into the presentation schedule so that the presentations remain in ascending order of `OrderNumber`. If a presentation with the same `OrderNumber` already exists, display “Duplicate `OrderNumber`” and do not insert the new presentation.
- `Summary()` that first calls the `display()` method for each `Presentation` object in the schedule, in ascending order of `OrderNumber`, followed by:
 - the total number of presentations;
 - the number of presentations for each category represented in the schedule. The categories may be displayed in any order.

For each of the sub-tasks, add a comment at the beginning of the code, using the hash symbol ‘#’, to indicate the sub-task the program code belongs to, for example:

```
In [1]: #Task 4.1
        Program code
```

Output:

Task 4.1

Write program code to declare the class `Presentation` and its methods.
 The constructor should initialise the attributes using the values of the parameters.
 Initially, `NextPresentation = None`.

Write program code to create **one** `Presentation` object and call its `display()` method to show the output. Use any arbitrary values of your choice for this object. [6]

Task 4.2

Write program code to declare the class `PresentationSchedule` and implement its methods. `AddPresentation()` must be recursive. [15]

Task 4.3

In a new cell, write program code to:

- initialise a new `PresentationSchedule`
- call `AddPresentation()` to insert each of the following **six** `Presentation` objects, in the order shown:

OrderNumber	TeamName	ProjectTitle	Category
3	Alpha	Smart light	Sustainability
1	Beta	AI maintenance	Digital Tech
5	NexRetail	AR shopping	Digital Tech
2	VitalScan	ChroniCare	Digital Tech
4	Microtrust	Credit scoring for SMEs	FinTech
2	EnviroGrid	Carbon tracing for maritime	Sustainability

- call the `Summary()` method and show the output. [3]

Save your Jupyter Notebook for Task 4.